

SENAI SUL

Disciplina: Teste de Sistemas

Professor: Daniel Goncalves Oliveira

Apresentação de Aula

Aluno: Rafael Soares, Gustavo Feliciano e Luis Guilherme

Turma: V2 Vespertino

Data: 05/05/2026

RELATÓRIO TÉCNICO: QUALIDADE E ESTRATÉGIAS DE TESTE DE SOFTWARE

Tópicos: Performance, Usabilidade/Acessibilidade e Segurança.

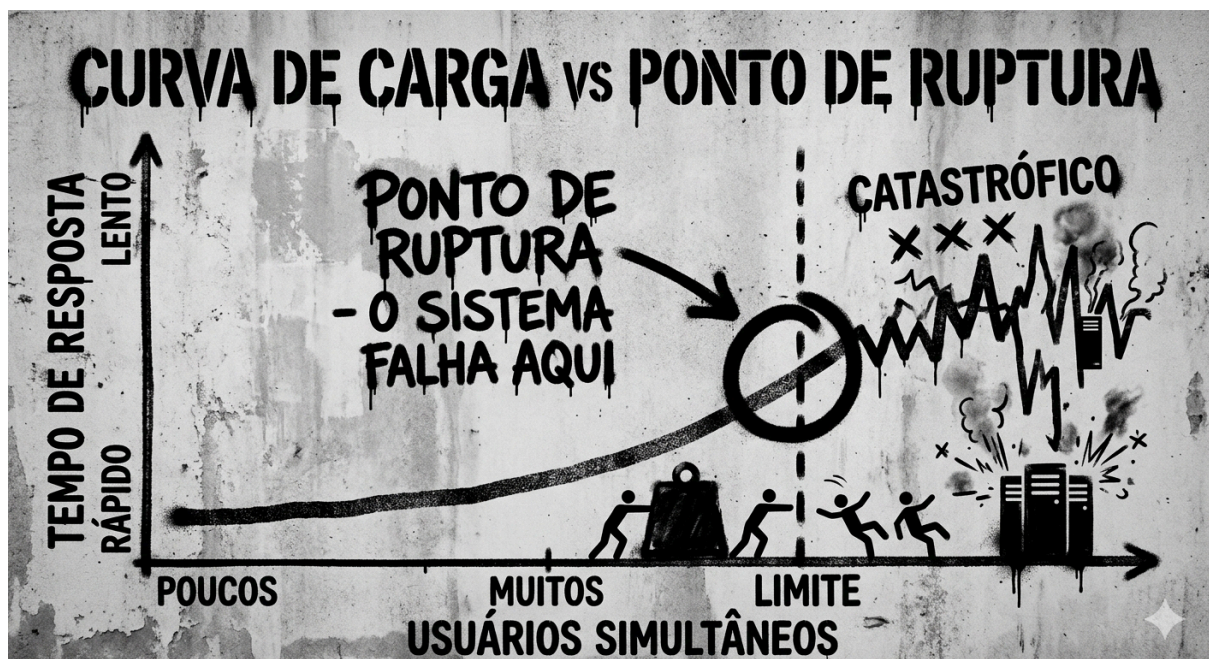
1. Testes Não Funcionais (Performance, Carga e Estresse)

A. Introdução

O Porquê: Os testes não funcionais são essenciais para **mitigar riscos de negócio**, evitando que lentidão ou quedas do sistema durante picos de acesso causem prejuízo financeiro e danos à reputação da marca.

O Como: Através da **automação** (ferramentas como JMeter e K6), simula-se carga massiva de usuários para monitorar métricas de hardware e latência, permitindo identificar e corrigir **gargalos (bottlenecks)** antes do lançamento.

B. Visualização Gráfica



O gráfico demonstra a curva onde o tempo de resposta aumenta conforme o número de usuários cresce, culminando no **Ponto de Ruptura**, onde o sistema para de responder adequadamente.

C. Questões para a Turma

1. Qual a diferença fundamental entre o objetivo de um teste de carga e um teste de estresse?
2. Como a latência de rede pode mascarar um gargalo que, na verdade, é de processamento de banco de dados?
3. O que acontece com o *Throughput* de um sistema quando ele atinge o seu ponto de saturação?
4. Por que um teste de performance deve ser executado em um ambiente que espelha a produção (staging)?
5. Como identificar se um vazamento de memória (*memory leak*) está ocorrendo durante um teste de resistência (*Soak Test*)?

D. Exemplos de Teste (Casos de Uso)

Cenário	Como Testar	Objetivo
Pico de Acessos	Simular a entrada súbita de 5.000 usuários em 10 segundos via K6.	Validar a elasticidade do servidor e o tempo de recuperação automática.
Uso Prolongado	Manter 500 usuários ativos realizando transações por 12 horas seguidas.	Identificar degradação de performance ou vazamentos de memória a longo prazo.
Ponto de Ruptura	Aumentar a carga continuamente até que o sistema retorne erro 500 ou trave.	Encontrar o limite máximo da arquitetura atual.

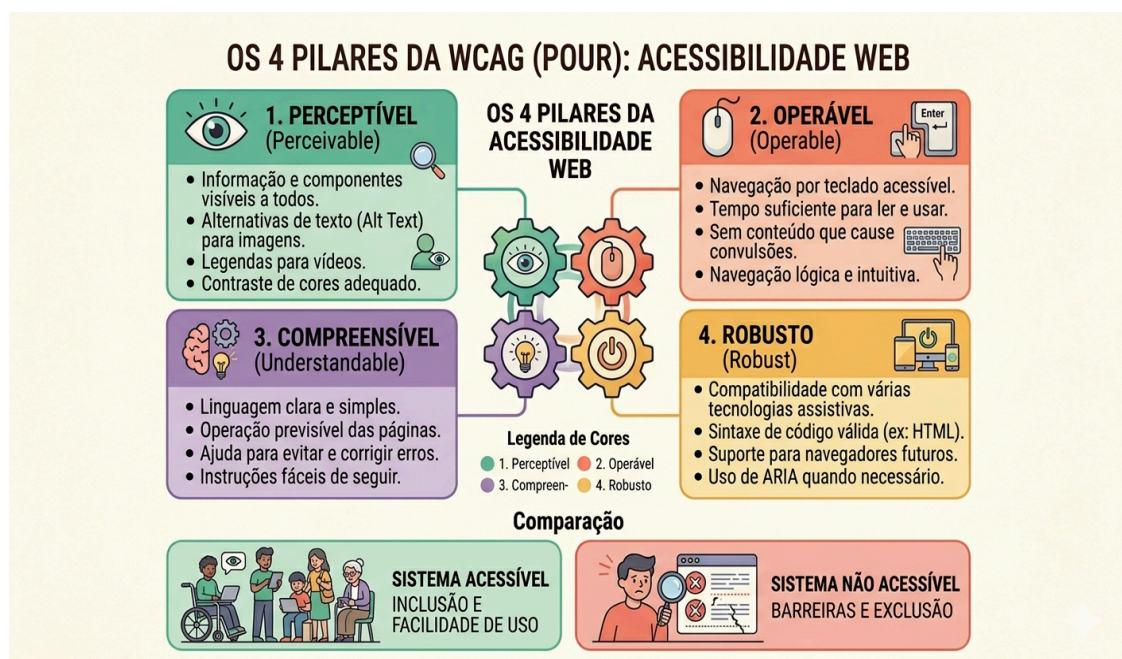
2. Testes de Usabilidade e Acessibilidade Digital (UX)

A. Introdução

O Porquê: Garante a **inclusão digital** e evita a perda de clientes por interfaces confusas, além de proteger a empresa contra penalidades legais e ampliar o alcance de mercado através da responsabilidade social e melhoria de SEO.

O Como: Aplica-se as **10 Heurísticas de Nielsen** e as diretrizes técnicas da **WCAG**, utilizando testes com usuários reais e ferramentas específicas (leitores de tela e validadores de contraste) para transformar a experiência em critérios objetivos de sucesso ou falha.

B. Visualização Gráfica



- **Perceptível** (Informação e interface visíveis).
- **Operável** (Interface navegável).
- **Understandable/Compreensível**.
- **Robusto** (Funciona em diversas tecnologias assistivas).

C. Questões para a Turma

1. Como a heurística de "Consistência e Padronização" auxilia na redução da carga cognitiva do usuário?
2. É possível um sistema ter excelente usabilidade, mas ser totalmente inacessível? Exemplifique.
3. Qual o papel técnico do atributo **alt** em imagens para usuários de leitores de tela?
4. Como o design responsivo impacta a acessibilidade para pessoas com deficiência motora?
5. Por que o feedback de erro deve ser informativo e não apenas visual (ex: apenas mudar a borda para vermelho)?

D. Exemplos de Teste (Casos de Uso)

Cenário	Como Testar	Objetivo
Navegação Assistiva	Percorrer todo o fluxo de compra utilizando apenas as teclas Tab e Enter .	Garantir operabilidade para pessoas que não podem usar o mouse.
Contraste e Leitura	Validar via plugin (ex: Wave) se o contraste texto/fundo atende ao nível AA da WCAG.	Garantir que daltônicos ou pessoas com baixa visão consigam ler o conteúdo.
Prevenção de Erros	Tentar submeter um formulário com dados inválidos e checar o feedback.	Verificar se o sistema ajuda o usuário a se recuperar de erros de forma intuitiva.

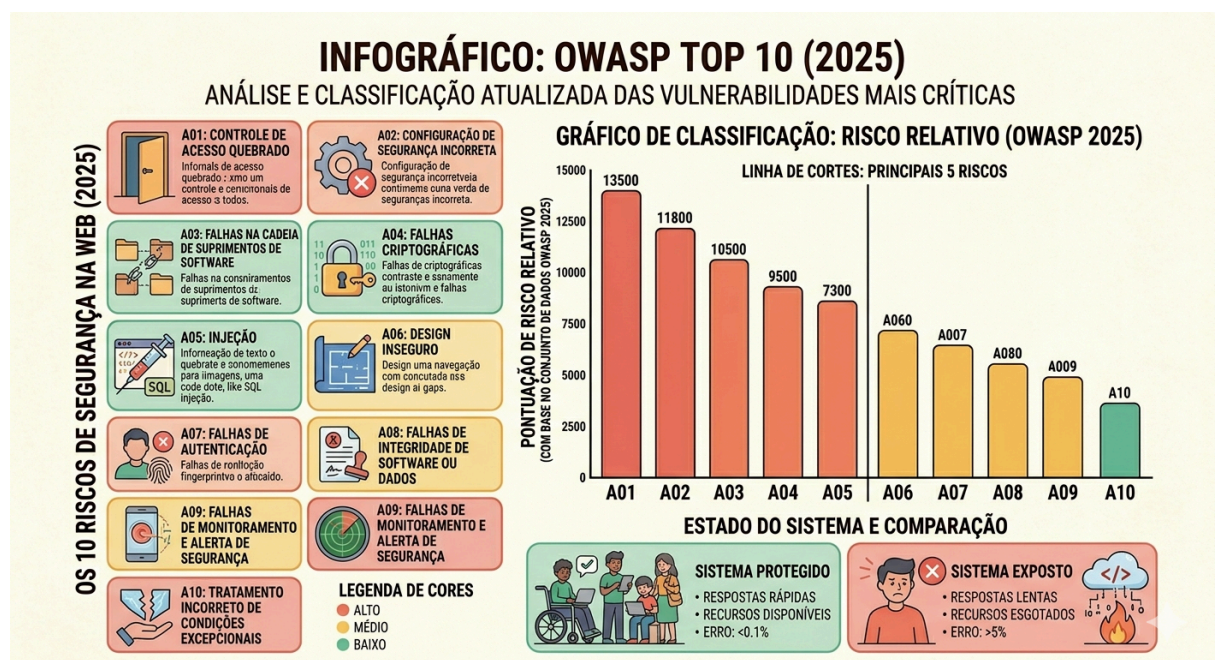
3. Fundamentos de Segurança da Informação nos Testes

A. Introdução

O Porquê: Visa transformar a proteção de dados em um requisito contínuo para evitar ataques cibernéticos e vazamentos que podem **destruir empresas**. Integrar isso ao desenvolvimento (**DevSecOps**) permite corrigir vulnerabilidades de forma precoce, reduzindo drasticamente os custos de reparo.

O Como: Utiliza-se o guia **OWASP Top 10** como referência para testes de análise estática (**SAST**), dinâmica (**DAST**) e invasão (**Pentest**). O foco é blindar entradas de dados contra injeções maliciosas e garantir que o controle de acesso proteja informações confidenciais.

B. Visualização Gráfica



1. Quebra de Controle de Acesso.
2. Falhas Criptográficas.
3. Injeção (SQL, XSS).

C. Questões para a Turma

1. Qual a diferença entre um ataque de SQL Injection e um Cross-Site Scripting (XSS)?
2. Por que a vulnerabilidade de "Broken Access Control" subiu para o topo do ranking da OWASP recentemente?

3. Como o princípio do "Menor Privilégio" ajuda a mitigar danos em caso de uma invasão?
4. Qual a limitação de uma ferramenta de varredura automática (DAST) em comparação ao Pentest manual?
5. Por que nunca devemos confiar em validações de dados feitas apenas no lado do cliente (Frontend)?

D. Exemplos de Teste (Casos de Uso)

Cenário	Como Testar	Objetivo
Injeção de Script	Inserir <code><script>alert('xss')</script></code> em um campo de cadastro e salvar.	Garantir que o sistema limpa (<i>sanitiza</i>) entradas de dados maliciosas.
Quebra de Acesso	Tentar trocar o ID do usuário na URL (ex: id=10 para id=11) para ver dados alheios.	Validar se o servidor checa a permissão de cada requisição individualmente.
Segurança de Login	Realizar múltiplas tentativas de login com senhas erradas em sequência.	Verificar se o sistema possui bloqueio de conta (<i>Rate Limit</i>) ou Captcha.

Fundamentos de Segurança da Informação

8. Projeto OWASP e o Top 10: O OWASP é uma fundação sem fins lucrativos focada na segurança de software. O "OWASP Top 10" é um documento de consenso que lista os riscos de segurança mais críticos para aplicações web, servindo como padrão para desenvolvedores e QAs priorizarem a correção de falhas que causam os maiores impactos financeiros e de imagem.